

Laporan Praktikum Mandiri 12

Penerapan Principal Component Analysis (PCA) untuk Reduksi Dimensi pada Dataset Breast Cancer Menggunakan Support Vector Machine

Nama Mahasiswa : Muhamad Faqih Rayya

Program studi : Teknik Informatika STT Terpadu Nurul Fikri, Depok

E-mail : 0110224212@student.nurulfikri.ac.id

Link Github : <https://github.com/faqihrayya0403/Machine-Learning>
: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

Link Kaggle

Abstrak

Dataset medis seperti Breast Cancer Dataset memiliki jumlah fitur yang cukup besar sehingga dapat meningkatkan kompleksitas model dan berpotensi menyebabkan overfitting. Oleh karena itu, diperlukan teknik reduksi dimensi untuk menyederhanakan data tanpa menghilangkan informasi penting. Penelitian ini bertujuan untuk menerapkan Principal Component Analysis (PCA) sebagai metode reduksi dimensi pada dataset Breast Cancer serta mengevaluasi pengaruhnya terhadap kinerja model klasifikasi Support Vector Machine (SVM).

1. Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Penjelasan: Library yang digunakan pada praktikum ini meliputi NumPy dan Pandas untuk pengolahan data, serta Matplotlib untuk visualisasi. Dataset Breast Cancer dimuat menggunakan `load_breast_cancer`. Pembagian data dilakukan dengan `train_test_split`, sedangkan `StandardScaler` digunakan untuk standarisasi data sebelum penerapan PCA. Model klasifikasi dibangun menggunakan Support Vector Machine (SVM), dan evaluasi performa model dilakukan dengan `accuracy score`, `classification report`, dan `confusion matrix`.

2. Mengakses File di Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Penjelasan

Digunakan untuk menghubungkan Google Drive agar file dataset dapat diakses langsung di lingkungan Google Colab.

3. Load Dataset Breast Cancer

```
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	v
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60	2019.0	0
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80	1956.0	0
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50	1709.0	0
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87	567.7	0
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20	1575.0	0

5 rows x 31 columns

Penjelasan: Pada tahap ini, dataset Breast Cancer dimuat menggunakan fungsi `load_breast_cancer` dari pustaka *scikit-learn*. Data kemudian dikonversi ke dalam bentuk **DataFrame** agar lebih mudah dianalisis, dengan setiap kolom merepresentasikan fitur dan satu kolom tambahan sebagai **label target**. Fungsi `df.head()` digunakan untuk menampilkan lima data teratas guna memastikan dataset berhasil dimuat dengan benar.

4. Eksplorasi Data

```
df.info()
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                                  Non-Null Count  Dtype
---  -
 0   mean radius                            569 non-null    float64
 1   mean texture                           569 non-null    float64
 2   mean perimeter                         569 non-null    float64
 3   mean area                             569 non-null    float64
 4   mean smoothness                        569 non-null    float64
 5   mean compactness                       569 non-null    float64
 6   mean concavity                         569 non-null    float64
 7   mean concave points                    569 non-null    float64
 8   mean symmetry                          569 non-null    float64
 9   mean fractal dimension                 569 non-null    float64
10   radius error                           569 non-null    float64
```

Penjelasan:

Perintah `df.info()` digunakan untuk menampilkan ringkasan struktur dataset, meliputi jumlah baris dan kolom, nama setiap fitur, tipe data, serta jumlah nilai non-null. Dari hasil ini dapat diketahui bahwa dataset Breast Cancer memiliki **569 data** dengan **31 kolom** (30 fitur dan 1 target) dan tidak terdapat nilai kosong, sehingga dataset siap digunakan untuk tahap analisis selanjutnya.

5. Pemisahan Fitur dan Label

```
X = df.drop('target', axis=1)
y = df['target']
```

Penjelasan: Pada tahap ini, dataset dipisahkan menjadi **fitur (X)** dan **label (y)**. Variabel X berisi seluruh fitur numerik dengan menghapus kolom target, sedangkan variabel y berisi kolom target yang merepresentasikan kelas data. Pemisahan ini dilakukan agar model Machine Learning dapat mempelajari pola dari fitur dan melakukan prediksi terhadap label yang ditentukan.

6. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

Penjelasan:

- X merupakan data fitur atau variabel independen, sedangkan y adalah data target atau label.
- test_size=0.2 berarti 20% dari total data digunakan sebagai data uji, dan 80% sisanya sebagai data latih.
- random_state=42 digunakan untuk memastikan pembagian data dilakukan secara konsisten setiap kali program dijalankan.
- stratify=y bertujuan menjaga proporsi kelas pada data latih dan data uji agar tetap seimbang sesuai dengan distribusi kelas pada data asli.

7. Standardisasi Data

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Penjelasan:

1. StandardScaler() digunakan untuk menstandarkan data dengan mengubah setiap fitur agar memiliki rata-rata (mean) = 0 dan simpangan baku (standar deviasi) = 1.
2. fit_transform(X_train) digunakan pada data latih untuk menghitung nilai mean dan standar deviasi, kemudian langsung menerapkan transformasi tersebut.
3. transform(X_test) digunakan pada data uji dengan parameter yang sama (mean dan standar deviasi dari data latih) agar tidak terjadi *data leakage*.

8. Model SVM Tanpa PCA (Baseline)

```
svm_no_pca = SVC(kernel='rbf', gamma='scale')
svm_no_pca.fit(X_train_scaled, y_train)

y_pred_no_pca = svm_no_pca.predict(X_test_scaled)

acc_no_pca = accuracy_score(y_test, y_pred_no_pca)
print("Akurasi SVM Tanpa PCA:", acc_no_pca)
print(classification_report(y_test, y_pred_no_pca))
```

Penjelasan:

Model dilatih menggunakan data yang sudah distandarisasi, kemudian digunakan untuk memprediksi data uji. Hasil prediksi dievaluasi menggunakan **akurasi** dan **classification report** untuk mengetahui performa model sebelum diterapkan PCA.

9. Penerapan PCA (3 Komponen)

```
pca = PCA(n_components=3)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

Penjelasan:

Kode ini digunakan untuk **mengurangi jumlah fitur** dengan metode PCA menjadi **3 komponen utama**.

PCA dilatih pada data latih yang sudah distandarisasi, lalu transformasi yang sama diterapkan pada data uji. Tujuannya adalah **menyederhanakan data**, mengurangi noise, dan meningkatkan efisiensi serta performa model.

10. Explained Variance Ratio

```
explained_var = pca.explained_variance_ratio_  
  
for i, var in enumerate(explained_var, start=1):  
    print(f"PC{i}: {var:.4f}")  
  
print("Total Variansi:", explained_var.sum())
```

```
PC1: 0.4441  
PC2: 0.1894  
PC3: 0.0954  
Total Variansi: 0.7290146639367362
```

Penjelasan:

1. `explained_variance_ratio_` menunjukkan kontribusi variansi tiap PC.
2. Loop mencetak variansi untuk PC1, PC2, PC3.
3. `explained_var.sum()` menampilkan total variansi yang dijelaskan oleh semua komponen PCA.

11. Model SVM dengan PCA

```
svm_pca = SVC(kernel='rbf', gamma='scale')  
svm_pca.fit(X_train_pca, y_train)  
  
y_pred_pca = svm_pca.predict(X_test_pca)  
  
acc_pca = accuracy_score(y_test, y_pred_pca)  
print("Akurasi SVM dengan PCA:", acc_pca)  
print(classification_report(y_test, y_pred_pca))
```

Penjelasan:

Kode tersebut digunakan untuk membangun **model SVM dengan PCA**. Data latih yang telah direduksi dimensinya menggunakan PCA digunakan untuk melatih model SVM, kemudian model memprediksi data

uji. Hasil prediksi dievaluasi menggunakan **akurasi** dan **classification report** untuk mengetahui performa model setelah penerapan PCA.

12. Visualisasi PCA 3D

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    X_train_pca[:, 2],
    c=y_train
)

ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_zlabel('PC3')
ax.set_title('Visualisasi PCA 3D - Breast Cancer')

plt.show()
```

Penjelasan:

Kode ini digunakan untuk memvisualisasikan hasil PCA dalam bentuk grafik 3D menggunakan tiga komponen utama (PC1, PC2, dan PC3). Setiap titik merepresentasikan satu data latih, sedangkan warna titik menunjukkan kelas berdasarkan label `y_train`. Visualisasi ini membantu melihat pola pemisahan kelas setelah reduksi dimensi dengan PCA.

13. Perbandingan Model



```
comparison = pd.DataFrame({
    'Model': ['SVM Tanpa PCA', 'SVM dengan PCA (3 Komponen)'],
    'Jumlah Fitur': [X_train.shape[1], X_train_pca.shape[1]],
    'Akurasi': [acc_no_pca, acc_pca]
})

comparison
```

Penjelasan:

Kode ini digunakan untuk **membandingkan performa model SVM tanpa PCA dan SVM dengan PCA**.

Tabel menampilkan **nama model**, **jumlah fitur yang digunakan**, dan **nilai akurasi** masing-masing model.

Perbandingan ini membantu melihat **pengaruh penggunaan PCA** terhadap kompleksitas data dan kinerja model.

Kesimpulan

Penerapan PCA berhasil mereduksi jumlah fitur pada dataset Breast Cancer tanpa mengurangi performa klasifikasi secara signifikan. Model SVM dengan PCA tetap menunjukkan akurasi yang baik, sehingga PCA efektif dalam menyederhanakan data dan meningkatkan efisiensi model.